

Genetic Algorithm-Based Optimization of Two Transfer Points (2TP) Problem in Random Graph

Julien Baudru

Université Libre de Bruxelles, Brussels, Belgium

Hugues Bersini

Université Libre de Bruxelles, Brussels, Belgium.

Abstract—This paper addresses the Two Transfer Point (2TP) problem, an important optimization challenge in vehicle routing and ridesharing scenarios. The objective is to identify optimal meeting and drop-off points in a graph to minimize the total travel distance for two users with distinct origin-destination pairs. We propose a Fitness Caching Genetic Algorithm (GA)-based approach that utilizes binary encoding, fitness evaluation based on difference of total travel distance, and evolutionary operations including tournament selection, crossover, and mutation. The algorithm is further enhanced with a fitness caching mechanism to improve efficiency. We conduct two sets of comparative experiments with an exact algorithm: the first under fixed generation budgets, and the second under equal time constraints, where the GA is only allowed the same execution time τ as the exact method. Results on complete random graphs show that the GA yields near-optimal solutions for small instances and acceptable approximations for larger graphs, even under limited time. Additionally, we evaluate the GA's performance on a real-world road network derived from OpenStreetMap data, demonstrating its practical applicability. In this context, the GA highlights its potential for scalable and time-efficient deployment. Our findings emphasize the trade-off between runtime and solution quality and validate the GA as a robust heuristic for the 2TP problem across both synthetic and realistic graph structures.

Index Terms—Two Transfer Point Problem, Vehicle routing, Ridesharing optimization, Genetic algorithm, Heuristic methods, Travel time minimization

I. INTRODUCTION

In this article, we start by defining the Two Transfer Point (2TP) problem, a routing optimization challenge in which two users with distinct start and destination points seek to minimize their combined travel distance by determining the optimal meeting and dropping points. We formalize the problem mathematically and provide an exact algorithm with quadratic time complexity to solve it as well as a genetic algorithm (GA) to find near-optimal solutions.

However, as the problem scales with larger graphs and more dynamic constraints, exact solutions become computationally prohibitive. To address this, we propose a Fitness Caching Genetic Algorithm (GA) that offers an efficient heuristic for solving the 2TP problem. To the knowledge of the authors, this is the first work to formally define and address the 2TP problem—minimizing the combined travel distance for exactly two users by optimizing both a meeting and a drop-off point—within a graph-based framework. This formulation distinguishes itself from general shared mobility problems by explicitly modeling transfer point optimization for two

users, supported by both an exact algorithm and a GA. The GA is not only capable of producing near-optimal solutions, but is also empirically shown to scale significantly better than the exact approach on large graphs. Furthermore, the proposed dual-method strategy allows for a comprehensive evaluation: the exact algorithm serves as a benchmark for solution quality, while the GA demonstrates practical viability for large instances where exact computation is infeasible, or very slow. This comparative analysis between solution quality and computational efficiency, tailored specifically to the 2TP problem, highlights the novelty and relevance of our approach in the context of scalable shared mobility planning.

This problem is part of the broader context of Shared Mobility Problems (SMP) [1], [2], which include carpooling, cab sharing and buspooling [1], [2], and have become a major trend in many urban cities around the world [1]. Trip sharing, as a problem 2TP, is intrinsically linked to vehicle routing and allocation challenges [2], [3], often formulated as variants of Vehicle Routing Problems (VRP) [4], [3], [1], [5], especially in dynamic and stochastic contexts [6], [7], [4], [8], [1], [3], [2]. The sources point out that ride-sharing systems are similar to other on-demand transportation services such as cabs and dial-a-ride services [7], [1]. Research in this area has explored a variety of resolution methods [1], [7], ranging from exact algorithms [1], [7], such as mixed integer programming or branch and bound [1], [7], [4], effective for small instances [1], [7] but limited due to non-polynomial complexity and scalability issues for larger problems [3], [1], [7], to heuristics and metaheuristics [6], [1], [7]. Among the latter, genetic algorithms (GA) are frequently used for their ability to quickly find good, albeit approximate, solutions for large-scale problems and in dynamic environments [4], [3], [7]. Techniques such as **clustering** [7], [3], [2], which involves grouping requests or journeys according to their spatio-temporal compatibility (including on the basis of shortest paths) [7], [3], are also used to improve the efficiency and scalability of solutions [7], [3], [2]. The evaluation of these approaches is often carried out using real data from large cities [8], [4], [3], [7], [2], simulations [8], [7], [2], or by comparing the results of heuristics with those of exact algorithms on small instances [4], [7].

The 2TP problem has diverse applications beyond transportation. For example, in collaborative robotics, coordinating meeting and transfer points for robots can optimize task efficiency and energy consumption. In supply chain management,

it can streamline the transfer of goods between warehouses to reduce costs. Additionally, in disaster response, identifying optimal meeting and dropping points for rescue teams and supplies can save critical time.

The remainder of this paper provides a formal problem definition, introduces an exact algorithm alongside a Fitness Caching GA, assesses the GA's error relative to the exact solution, and evaluates the performance of both approaches both theoretically and empirically. The evaluation is conducted on randomly generated complete graphs as well as on small real-world road networks.

II. TWO TRANSFER POINT (2TP) PROBLEM

Given a graph $G(V, E)$ and two paths $a-c$ and $b-d$ sharing a common segment $m-k$, the goal is to find the meeting node m and the drop-off node k , such that the pair of nodes m, k minimizes the sum of the parameters of the edges $e \in E$ of all the shortest paths SP included in this scenario. Figure 1 illustrates this problem graphically, and we can formalize this problem as in Definition II.1.

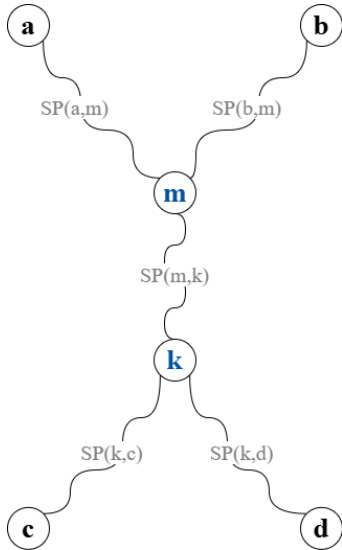


Figure 1: Two Transfer Point (2TP) problem in general graph.

Definition II.1 (2TP Problem). Given a graph $G(V, E)$, two paths $a-c$ and $b-d$ sharing a common segment $m-k$, with $a, b, c, d \in V$, $SP(u, v)$ the shortest path between $u, v \in V$ and $\tau(u, v)$ the sum of the parameters of the edges $e \in SP(u, v)$, is there exists a pair $m, k \in V$ such that:

$$(m, k) = \min_{m, k} (\tau(a, m) + \tau(m, k) + \tau(k, c) + \tau(b, m) + \tau(m, k) + \tau(k, d))$$

In a real-world context, the sum of the edge parameters will generally be the cumulative travel time for each edge or the total travel distance, in the remainder of this article, we will use the distance parameter.

The pseudo-code (Algorithm 1) gives an exact solution for solving the 2TP problem in a general graph with an approximate worst-case time complexity of $O(n^2)$ where $n = |V|$ (cf. Section IV-C). This algorithm begins by precomputing and caching all-pairs shortest path distances using Dijkstra's

algorithm [9]. It then iterates over all possible pairs of meeting node m and dropping node k in the graph. For each pair, it retrieves the relevant shortest path distances from the cache to compute the total travel distance for both users. The algorithm keeps track of the minimum total distance, and at the end, it returns the optimal pair of nodes that minimizes this distance.

Algorithm 1 Exact Algorithm for the Two Transfer Point Problem with Precomputed Distances

Require: Graph G with $|V|$ nodes, sources nodes a, b , targets nodes c, d

Ensure: Optimal meeting and dropping points

- 1: Precompute and cache all-pairs shortest path distances using Dijkstra for all node pairs in G
 - 2: Store the distances in a dictionary-like structure: $\text{dist}[u][v] = \text{shortest path distance from } u \text{ to } v$
 - 3: Initialize $\text{min}_{\text{sum}} \leftarrow \infty$
 - 4: **for** each node $m \in G$ **do**
 - 5: **for** each node $k \in G$, where $k \neq m$ **do**
 - 6: $\text{sum}_m \leftarrow \text{dist}[a][m] + \text{dist}[b][m]$
 - 7: $\text{sum}_k \leftarrow \text{dist}[m][k]$
 - 8: $\text{sum}_e \leftarrow \text{dist}[k][c] + \text{dist}[k][d]$
 - 9: $\text{sum} \leftarrow \text{sum}_m + \text{sum}_k + \text{sum}_e$
 - 10: **if** $\text{sum} \leq \text{min}_{\text{sum}}$ **then**
 - 11: $\text{min}_{\text{sum}} \leftarrow \text{sum}$
 - 12: **end if**
 - 13: **end for**
 - 14: **end for**
 - 15: **return** min_{sum}
-

We hypothesize that the 2TP problem is 3SUM-hard, implying that there is no sub-quadratic algorithm for this problem. An algorithm is *sub-quadratic* if there exists an $\epsilon > 0$ such that the algorithm runs in time $O(n^{2-\epsilon})$. The 3SUM problem involves determining whether there exist three numbers within a given set that sum to zero. We can formalize this problem as in the Definition II.2.

Definition II.2 (3SUM Problem). Given a set S of n integers, does there exist $x, y, z \in S$ such that $x + y + z = 0$?

The 3SUM problem can be solved using a $O(n^2)$ algorithm¹ and it is generally accepted that $O(n^2)$ is the worst case lower bound as expressed in Conjecture II.1.

Conjecture II.1 (3SUM conjecture). *There is no sub-quadratic algorithm for the 3SUM problem [10].*

However, this conjecture has been refuted in the meantime by proving better lower bound [11], [12], [13]. While a formal reduction from 3SUM to 2TP, or vice versa, is not presented in this work, we hypothesize that the 2TP problem in general graphs is 3SUM-HARD. However, this complexity may not extend to road graphs. Road networks differ significantly from classical graphs and often exhibit structured properties, such as a low Highway Dimension [14], which can be leveraged by specialized algorithms like Contraction Hierarchies (CH)

¹By first sorting the S set, then testing each 3-tuples

[15]. These methods are highly efficient for standard shortest-path problems on static graphs, offering rapid query performance after substantial preprocessing. Nevertheless, CH and similar techniques may falter in scenarios involving dynamic changes, complex constraints (e.g., time windows, multi-modal transport), or custom routing preferences, situations where preprocessing becomes infeasible or ineffective. In such contexts, more flexible approaches like Genetic Algorithms (GAs) may prove advantageous despite their higher computational overhead. While GAs may not outperform classical algorithms in purely polynomial settings, their adaptability makes them suitable for the richer, more dynamic versions of the problem we ultimately aim to address. In the following, our focus will be directed towards classical graphs, with particular emphasis on complete graphs, due to their high connectivity, which inherently results in higher complexity.

III. FITNESS CACHING GENETIC ALGORITHM

GAs are a class of optimization techniques inspired by the principles of natural selection [16]. GAs operate by evolving a population of candidate solutions through selection, crossover, and mutation processes, guided by a fitness function. These algorithms are particularly effective for solving complex optimization problems where conventional methods may be computationally impractical or inefficient [17].

In the context of 2TP problem, we employ a Fitness Caching GA to optimize the transfer points, aiming to minimize the total travel distance for both users. As before, the 2TP problem is modeled using a complete random graph $G(V, E)$ with $|V| = N$, in our experiments the number of nodes N is chosen from $\{10, 25, 50, 100\}$. The pseudo-code (Algorithm 2) provides the details of the steps in the GA process. Two users are assigned random start and destination nodes, ensuring distinct locations. The algorithm iteratively evolves candidate solutions to identify the pair of nodes, m and $k \in V$, that results in the most efficient routing for the users. Solutions are encoded as binary strings, representing the indices of the meeting and dropping points, $m, k \in V$, details are given in the Section III-A.

Fitness is computed as the negative total travel distance for both users, ensuring minimization by maximizing fitness values. The algorithm employs tournament selection for parent selection, bit-wise crossover for offspring generation, and bit-flip mutation with a 10% probability to maintain diversity. The population evolves over a predefined number of generations T , with the best-performing individuals retained, this elitist strategy prevents the loss of promising candidates and accelerates convergence toward the (near) optimal solution.

It is true that the number of possible (m, k) pairs in this problem is limited to $|V|^2$, and for small values of $|V|$, enumerating all solutions may appear tractable. However, our GA implementation leverages a *fitness caching mechanism* that stores the evaluation results of previously seen individuals, thereby avoiding redundant computation and enabling the algorithm to focus resources on exploring new regions of the solution space. In contrast to brute-force enumeration—which exhaustively and indiscriminately evaluates all

combinations—the GA efficiently guides the search using evolutionary pressure toward more promising regions of the search space. Moreover, enumeration lacks the adaptive sampling and probabilistic exploration mechanisms that allow GAs to generalize across different problem settings and scale more naturally to larger instances. As a result, even when the number of evaluations is on par with the number of total possible solutions, the GA often discovers better solutions faster and with fewer redundant checks.

Algorithm 2 Genetic Algorithm with Fitness Caching for the Two Transfer Point Problem

Require: Graph G with $|V|$ nodes, source nodes a, b , target nodes c, d , population size P , number of generations T

Ensure: Optimal meeting and dropping points

```

1: Initialize population  $P_0$  with random binary-encoded solutions
2:  $fitness\_cache \leftarrow \emptyset$ 
3:  $best\_fitness \leftarrow -\infty$ 
4:  $best\_solution \leftarrow \emptyset$ 
5: function EVALUATEWITHCACHE( $individual$ )
6:   if  $individual \in fitness\_cache$  then
7:     return  $fitness\_cache[individual]$ 
8:   else
9:      $fitness \leftarrow \text{ComputeFitness}(individual)$ 
10:     $fitness\_cache[individual] \leftarrow fitness$ 
11:    return  $fitness$ 
12:   end if
13: end function
14: for  $t \leftarrow 1$  to  $T$  do
15:   for each  $individual$  in population do
16:      $individual.fitness \leftarrow \text{EVALUATEWITH-}$ 
17:        $\text{CACHE}(individual)$ 
18:   end for
19:   Select parents using tournament selection
20:   Generate offspring using crossover and mutation
21:   for each  $child$  in offspring do
22:      $child.fitness \leftarrow \text{EVALUATEWITHCACHE}(child)$ 
23:   end for
24:   Merge offspring into the population
25:   Retain the top  $P$  individuals based on fitness
26:   if fitness of best individual in population  $>$ 
27:      $best\_fitness$  then
28:     Update  $best\_fitness$  and  $best\_solution$ 
29:   end if
30: end for
31: return  $best\_solution$  with corresponding  $best\_fitness$ 

```

A. Encoding

Solutions in the genetic algorithm are encoded as fixed-length binary strings that represent the decision variables of the 2TP Problem, specifically the indices of the meeting and dropping points, $m, k \in V$. Each of these points is encoded using a fixed number of bits, typically one byte, resulting in a total individual length of 16 bits. This design constrains the maximum number of nodes to 256 (2^8), which is sufficient

for our experiments; however, the encoding length can be increased to support larger networks.

The structure of the binary string is divided evenly: the first half encodes the meeting point and the second half encodes the dropping point. These binary segments are interpreted as unsigned integers and mapped to valid node indices using the modulo operation, ensuring all decoded values correspond to existing nodes in the graph regardless of the raw binary value. This encoding strategy allows efficient manipulation and crossover operations at the bit level. A simple example of this encoding scheme is illustrated in Figure 2, where each individual represents a unique pair of terminal points in the network.

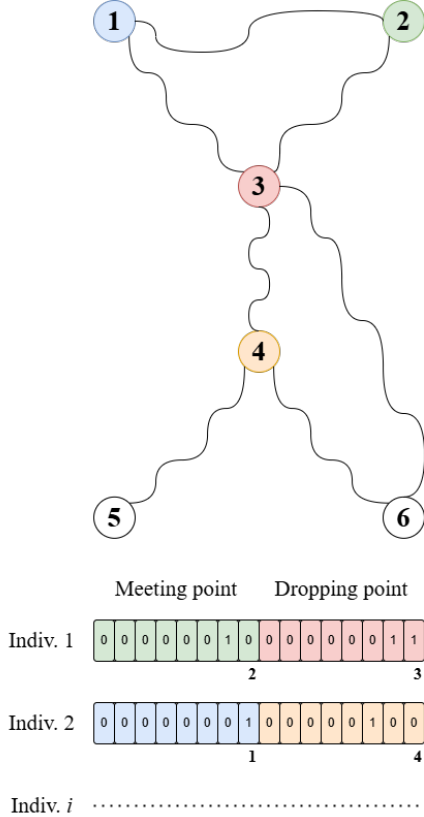


Figure 2: Two Transfer Point (2TP) individuals encoding.

B. Selection

The genetic algorithm employed in our experiments uses *tournament selection* to choose parents for crossover. This method involves randomly selecting a subset of individuals from the current population and choosing the one with the highest fitness among them to act as a parent. In the implementation, the tournament size k is defined as $\max(2, \lfloor \frac{P}{10} \rfloor)$, where P is the population size. This dynamic sizing ensures a reasonable balance between selection pressure and population diversity, adapting to small or large populations alike.

Tournament selection is particularly suited for problems like the 2TP problem, where the fitness landscape may contain numerous local optima and sharp discontinuities. By selecting the best individual from a random subset rather than directly

based on proportional fitness, tournament selection reduces the risk of premature convergence caused by overly dominant individuals early in the search. This approach introduces a moderate degree of stochasticity, maintaining diversity in the gene pool while still favoring individuals with higher fitness. In contrast to more deterministic or proportional methods, tournament selection does not require fitness normalization and is less sensitive to the scale of the fitness values. This makes it robust in scenarios where raw fitness differences may be large or unevenly distributed across the population.

Alternative selection mechanisms such as roulette wheel selection and rank-based selection also exist, each with different trade-offs. Roulette wheel selection assigns selection probabilities directly based on relative fitness values, favoring individuals with high fitness but risking early loss of genetic diversity. Rank-based selection, on the other hand, assigns probabilities based on the ranking of individuals rather than their absolute fitness, providing smoother control over selection pressure. Although this implementation does not explicitly name elitism, an elitist strategy is effectively incorporated by preserving the top P individuals after each generation. Following the generation of a new offspring via crossover and mutation, the population is augmented and then truncated by sorting all individuals by fitness and retaining only the best-performing subset. This ensures that the fittest solutions persist across generations, guiding the evolutionary process steadily toward optimal or near-optimal regions of the search space.

C. Fitness

The fitness function evaluates the quality of solutions in the GA by calculating the total travel distance for a pair of meeting and dropping points. Invalid solutions, where the meeting and dropping points are identical, are penalized with $-\infty$. For valid solutions, the fitness is the negated sum of shortest path distances, using Dijkstra, for users traveling to the meeting point ($a - m$, $b - m$), between meeting and dropping points ($m - k$), and from the dropping point to their destinations ($k - c$, $k - d$). This ensures the algorithm maximizes fitness while minimizing total travel distance.

IV. RESULTS

In this section, we present the results of our experiments evaluating the performance of the GA for solving the 2TP problem. We demonstrate that the population converges to a near-optimal solution over successive generations, highlighting the algorithm's effectiveness in minimizing the total travel distance for the users. The results also include a practical example using a real-world road network, illustrating the GA's applicability to real-world scenarios. Additionally, we compare the solutions obtained by the GA (Algorithm 2) with the solutions obtained by the exact algorithm (Algorithm 1) on graphs with $|V| \in \{10, 25, 50, 100\}$, where each edge weight is a floating-point distance δ randomly drawn from the interval $[1, 10]$. In both experiments we repeated 50 random experiments with a fixed seed to enable reproducibility, the code and results are available on a [GitHub repository]. The complete random graph structure ensures all possible routes

and transfer points are represented, enabling a comprehensive evaluation of the algorithm while also reflecting the worst-case runtime scenario, as real-world road networks are typically not fully connected. In the first set of experiments of the errors, the GA operates with a fixed generation budget, while in the second, it is allocated the same time budget as the exact algorithm. Finally, we analyze the running time of both algorithms and provide insights into 10 instances of real world scenarios where the GA may be preferred over the exact solution, particularly in larger or more complex instances.

A. Convergence

In this section, we evaluate the performance of each GA instance using the 50 instance of complete random graphs $G(V, E)$, where $|V| = 100$. The GA instances are tested with population sizes $P \in \{5, 10, 20, 50\}$ and number of generations $T \in \{250, 500\}$. The corresponding results are presented in Figures 3, and 4. In the following results, it is important to note that the total distance difference between two users cannot be zero, as they are required to meet at a common intersection (node m) and subsequently separate at a different intersection (node k).

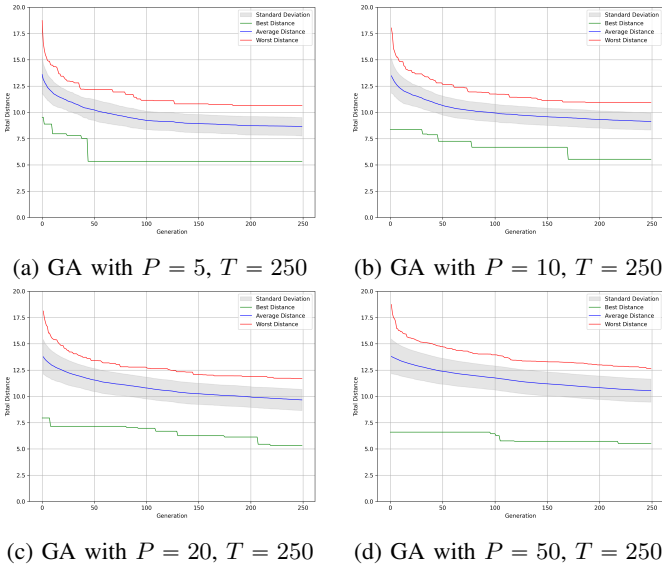


Figure 3: Evolution of the average fitness function over 50 instances of complete random graphs with $|V| = 100$ nodes for two GA configurations with $T = 250$.

Figure 3 presents a comparative analysis of various population sizes over the course of 250 generations. The results, averaged over 50 independent runs, indicate that smaller population sizes tend to converge more rapidly toward an optimal or near-optimal solution. Specifically, after 250 generations, the population with size $P = 5$ achieves an average distance difference of ≈ 7.75 , whereas the population with size $P = 50$ attains an average distance difference of ≈ 11 . Moreover, the optimal solution is identified as early as generation ≈ 45 for $P = 5$, while it remains undiscovered within the 250-generation limit for $P = 50$.

Figure 4 illustrates a comparative analysis of different population sizes over 500 generations. Consistent with the

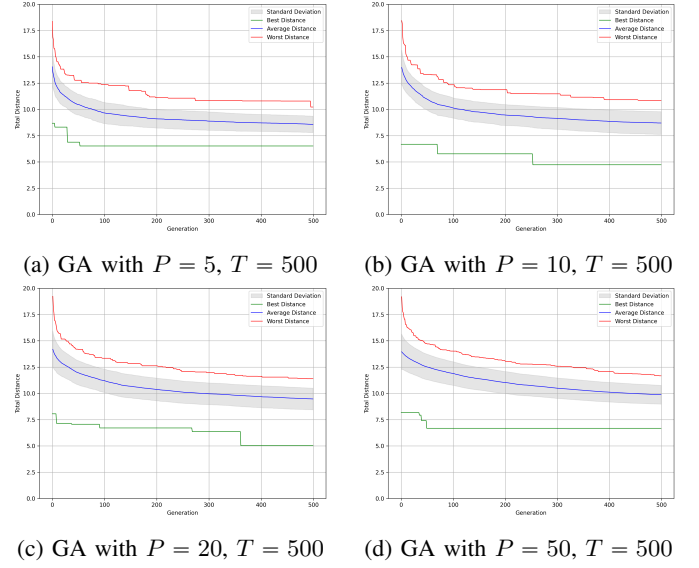


Figure 4: Evolution of the average fitness function over 50 instances of complete random graphs with $|V| = 100$ nodes for two GA configurations with $T = 500$.

previous observations, and despite the doubled computational budget, smaller population sizes tend to converge more rapidly, on average (across 50 independent instances), toward optimal or near-optimal solutions. However, this advantage is counterbalanced by an increased risk of premature convergence, which can lead to sub-optimal outcomes. For instance, the best solution obtained by the population with size $P = 5$ (Figure 4a) remains stagnated at a best distance difference of 6.52. In contrast, the population with size $P = 10$ (Figure 4b) is able to further improve, reaching a solution with a distance difference of 4.73.

The slower convergence occurs because larger populations provide greater genetic diversity, which increases the search space and delays premature convergence. While this delay can result in slower progression towards the optimal solution, it also reduces the likelihood of becoming trapped in suboptimal solutions, highlighting a trade-off between exploration and convergence speed.

B. Mean Relative Error

In this section, we evaluate the solutions obtained by the GA (Algorithm 2) with different size of population to the solutions obtained by the exact algorithm (Algorithm 1) on the same 50 random instance of complete random graphs with $|V| \in \{10, 25, 50, 100\}$, first with a fixed budget of generation for the GA and then with an execution time budget equal to the one required by the exact algorithm.

The Mean Relative Error (MRE) is calculated by first determining the relative error for each instance. This is defined as the percentage difference between the distance obtained by the GA and the exact solution, normalized by the exact solution's distance. The MRE is then computed as the arithmetic mean of these relative errors across all evaluated instances, providing a measure of the GA's average deviation from the optimal

solution. In the following experiment, we used the *normalized Kernel Density Estimation (KDE)* to visualize the distribution of MRE values for different parameter configurations. KDE is a non-parametric method for estimating the probability density function of a random variable, which provides a smooth approximation of the underlying data distribution. By applying KDE to the MRE values and normalizing the resulting densities, we ensure consistent scaling across configurations, enabling clear visual comparisons. Curves are plotted over a common axis to reveal shifts in error distributions, where tighter, left-shifted peaks indicate better performance and broader, right-shifted curves signify higher relative error.

1) *MRE when fixed generation budget*: In this experiment, each population size was given the same generation budget, namely $T = 500$, and was evaluated on the same 50 randomly generated instances of complete graphs.

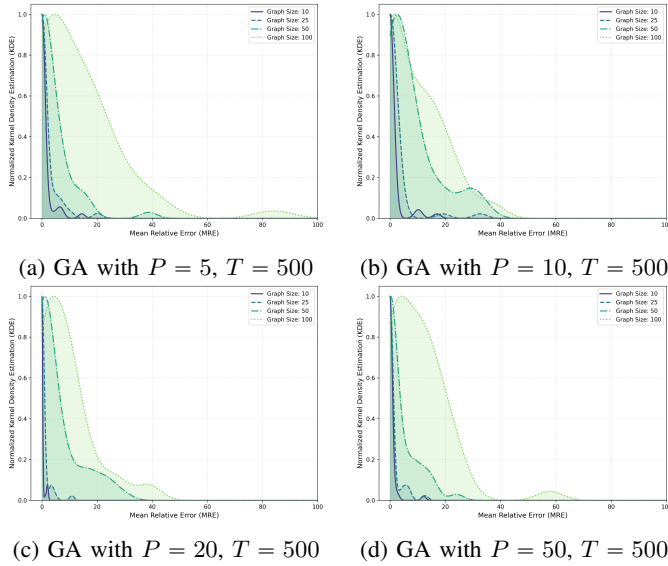


Figure 5: Normalized Kernel Density Estimation (KDE) of the Mean Relative Error (% MRE) for GA with fixed number of generations $T = 250$ and variable population size P over 50 instances of complete random graphs.

The first clear observation from Figure 5 is that, for a fixed population size P , the MRE tends to increase with the size of the graph, i.e., with the number of nodes and edges. This behavior is expected, as larger graphs imply a larger search space, making the optimization problem more challenging.

We also observe that for graph sizes $|V| \in \{10, 25, 100\}$, the population size $P = 20$ achieves the best average performance in terms of MRE, as shown in Figure 5c. In contrast, for graphs of size $|V| = 50$, the population size $P = 50$ yields the lowest average MRE, as illustrated in Figure 5d.

Given that the population size $P = 20$ achieves the lowest average Mean Relative Error (MRE), we proceed to evaluate the impact of the number of generations T on performance, using the same set of randomly generated graph instances.

Figure 6 demonstrates that, for a fixed population size and across all tested graph sizes, increasing the number of generations T systematically leads to a lower average MRE. This trend reflects the fact that a greater number of generations

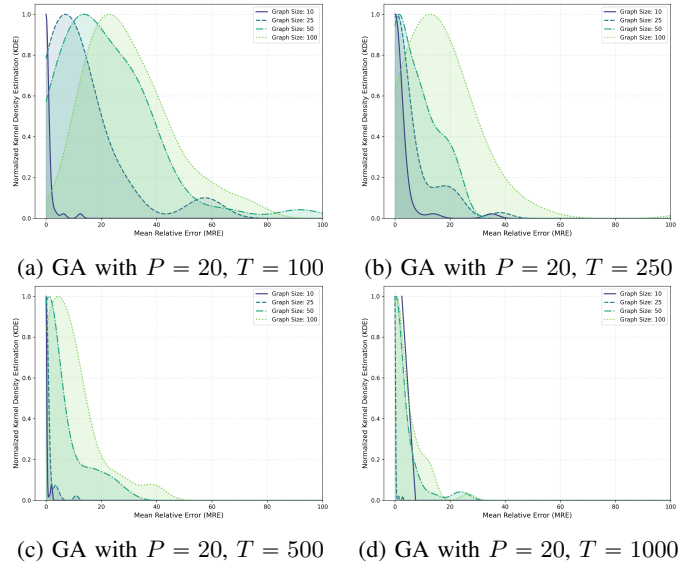


Figure 6: Normalized Kernel Density Estimation (KDE) of the Mean Relative Error (% MRE) for GA with fixed population size $P = 20$ and variable number of generations over 50 instances of complete random graphs.

allows the algorithm more opportunity to converge toward the optimal solution. These findings are consistent with the behavior of the fitness function observed in Section IV-A. Furthermore, since the same set of random graph instances was used in both experiments, the results presented in Figure 5c and Figure 6c are identical.

2) *MRE when τ generation budgets*: Following the methodology of the previous experiment, we evaluate the performance of the GA (Algorithm 2) with varying population sizes by comparing its solutions to those obtained by the exact algorithm (Algorithm 1) across the same set of 50 randomly generated complete graphs, with vertex counts $|V| \in \{10, 25, 50, 100, 200, 400\}$. In contrast to the prior setup, the GA is now allotted a runtime τ equal to the execution time required by the exact algorithm for each instance. Consequently, for smaller graph sizes (where the exact algorithm requires less computational time), the GA is provided with proportionally less time, potentially limiting its ability to achieve convergence. Table I reports the runtime τ required by the exact algorithm (EA) for each tested size $|V|$ of the randomly generated complete graphs.

Table I: Average running time of the the Exact Algorithm (EA) on the complete random graphs

Graph Size — V —	EA (τ)
10	0.001s
25	0.003s
50	0.020s
100	0.157s
200	1.239s
400	11.675s

It is important to note that in this experiment, when the allocated time τ was insufficient for the GA to complete a full evolutionary cycle, the first solution encountered was retained.

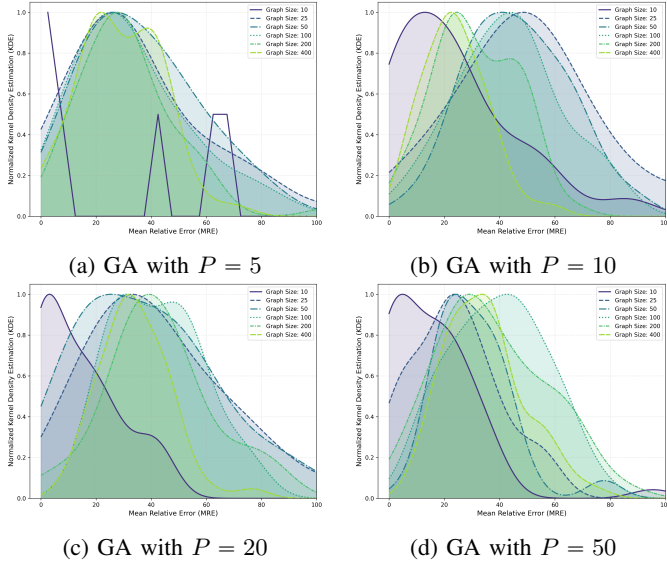


Figure 7: Normalized Kernel Density Estimation (KDE) of the Mean Relative Error (% MRE) for GA with T linked to τ and variable population size P over 50 instances of complete random graphs.

This situation occurred, for instance, with graphs of size $|V| = 10$, and accounts for the irregular behavior observed in the curves shown in Figure 7a.

For small graph instances, the likelihood of identifying relatively good solutions in the initial iterations is higher. This may explain why all GA variants depicted in Figure 7 exhibit a low average MRE for graphs with $|V| = 10$.

Moreover, the GA configured with a population size of $P = 5$ exhibits the lowest variance in MRE across all graph sizes, see Figure 7a. This behavior may be attributed to the reduced computational overhead associated with smaller populations, allowing the algorithm to converge more quickly, regardless of the optimality of the solution. In contrast, the GA with a population size of $P = 50$ appears to perform relatively well under the imposed time constraint, see Figure 7d, which could be explained by the enhanced exploratory capability afforded by its larger population size.

Finally, for larger instances, specifically when $|V| = 400$ in the complete random graphs, the GA configured with a population size of $P = 10$, $P = 20$ demonstrates superior performance and appears to mitigate premature convergence, as illustrated in Figure 7b and 7c.

C. Real Road Network Case

In this experiment, instead of using completely random graphs, we evaluate the performance of the GA (Algorithm 2) with varying population sizes by comparing its solutions to those obtained by the exact algorithm (Algorithm 1) on 10 randomly generated instances derived from a single real-world road network. This network comprises $|V| = 2390$ vertices and $|E| = 5296$ edges. Unlike the previously tested random graphs, the real-world road network is not complete. The data were extracted from OpenStreetMap, and the encoding length

was extended to 25 bits to accommodate the OpenStreetMap node identifiers.

The exact algorithm (Algorithm 1) systematically evaluates all possible pairs of meeting and dropping points in the graph. Its complexity can be expressed as:

$$O(|V|^2) + O(|E| + |V| \log |V|)$$

where $O(|E| + |V| \log |V|)$ is the time complexity of Dijkstra's algorithm for a graph with $|V|$ nodes and $|E|$ edges. Since Dijkstra's algorithm is only once at the beginning, the complexity of The exact algorithm (Algorithm 1) can therefore be simplified into:

$$O(|V|^2)$$

the overall complexity scales quadratically with $|V|$, making the exact algorithm computationally expensive as the graph size increases. The GA is therefore particularly advantageous for very large graph instances. While the exact algorithm provides guaranteed optimal solutions, its computational cost becomes prohibitive for large graphs due to its dependence on the quadratic scaling of node pairs.

1) *MRE when fixed generation budget:* Tables II, III, IV, and V present the execution times obtained on the small real-world road network for each tested population size $P \in \{5, 10, 20, 50\}$ and for each $T \in \{100, 250, 500, 1000\}$, along with the execution time τ of the exact algorithm, which remains constant across all tables.

Table II: Average running time of the the Exact Algorithm (EA) and GA ($P = 5$) on a small real road network

$G = (V , E)$	EA (τ)	GA ($T = 100$)	GA ($T = 250$)	GA ($T = 500$)	GA ($T = 1000$)
2390, 5296	10.52s	0.92s	2.24s	4.34s	8.60s

Table III: Average running time of the the Exact Algorithm (EA) and GA ($P = 10$) on a small real road network

$G = (V , E)$	EA (τ)	GA ($T = 100$)	GA ($T = 250$)	GA ($T = 500$)	GA ($T = 1000$)
2390, 5296	10.52s	0.95s	2.21s	4.41s	8.66s

Table IV: Average running time of the the Exact Algorithm (EA) and GA ($P = 20$) on a small real road network

$G = (V , E)$	EA (τ)	GA ($T = 100$)	GA ($T = 250$)	GA ($T = 500$)	GA ($T = 1000$)
2390, 5296	10.52s	1.06s	2.40s	4.52s	8.89s

Table V: Average running time of the the Exact Algorithm (EA) and GA ($P = 50$) on a small real road network

$G = (V , E)$	EA (τ)	GA ($T = 100$)	GA ($T = 250$)	GA ($T = 500$)	GA ($T = 1000$)
2390, 5296	10.52s	1.39s	2.79s	5.13s	9.39s

As anticipated based on previous results, the execution time of the GA increases with the population size for a fixed generation budget. Similarly, for a fixed population size, an increase in the generation budget leads to a longer execution time. Thus, the runtime of the GA is directly influenced by both the population size and the number of generations. Notably, even under the most computationally intensive configuration

tested—i.e., a population size of $P = 50$ and a generation budget of $T = 1000$ (see Table V)—the GA’s execution time remains lower than that of the exact algorithm on the small real-world road network.

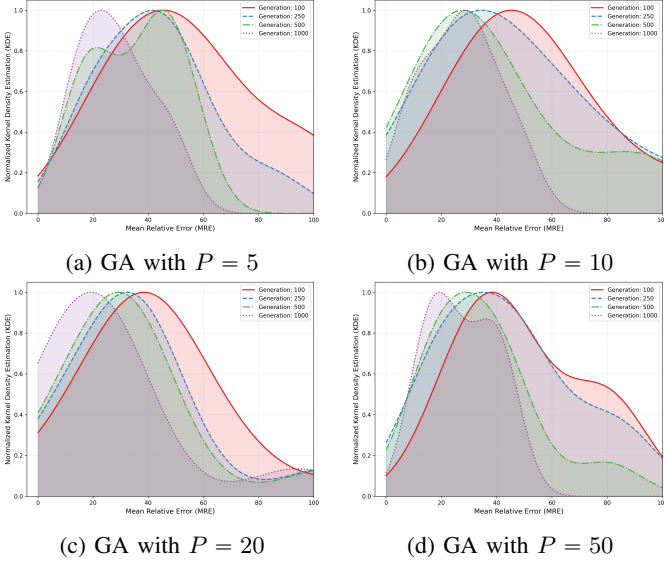


Figure 8: Normalized Kernel Density Estimation (KDE) of the Mean Relative Error (% MRE) for GA with variable number of generations and population size P over 10 instances of a real world road network with $|V| = 2390$ and $|E| = 5296$.

Figure 8 shows that, across all tested population sizes, the GA benefits from an increased number of generations T in terms of MRE. Furthermore, among the population sizes considered, the configuration with $P = 20$, see Figure 8c, exhibits the lowest variance in MRE with respect to the number of generations T , indicating more consistent performance. Notably, when this configuration is allocated a generation budget of $T = 1000$, the MRE in terms of distance stabilizes around 19% in real world application.

Although the GA provides approximate solutions, it does so significantly faster than the exact algorithm, with its runtime primarily influenced by the P and T . This highlights an inherent trade-off between computational efficiency and solution accuracy. By appropriately tuning these parameters, the GA can effectively balance execution time and solution quality, making it a practical and scalable method for large-scale problems where exact solutions are computationally infeasible.

2) *MRE when τ generation budgets*: Similar to the experiments conducted on random graphs, and based on the best results obtained under fixed generation budgets, we evaluated the performance of the Genetic Algorithm (GA) (Algorithm 2) with varying population sizes by comparing its solutions to those produced by the exact algorithm (Algorithm 1), under the constraint that the GA was allotted a runtime τ equal to the execution time required by the exact algorithm for each instance.

As shown in Figure 9, the GA configured with a population size of $P = 20$ achieves the lowest average MRE, approximately 17%, when executed under the same time budget as the

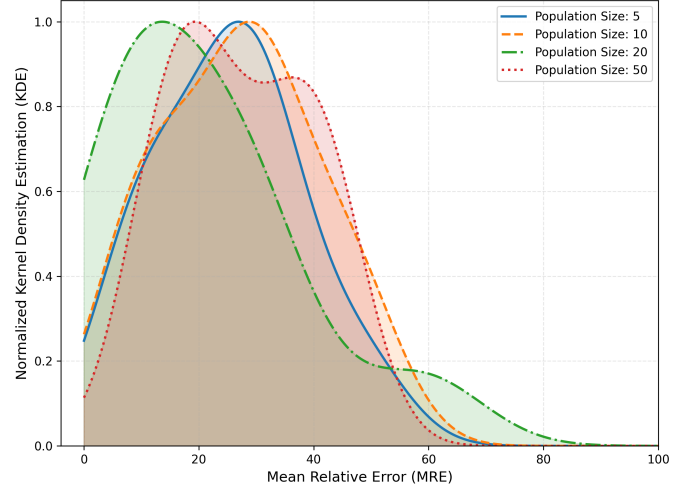


Figure 9: Normalized Kernel Density Estimation (KDE) of the Mean Relative Error (% MRE) for GA with T linked to τ and variable population size P over 10 instances of a real world road network with $|V| = 2390$ and $|E| = 5296$.

exact algorithm. This configuration is followed in performance by the GA with $P = 50$.

V. FUTURE WORKS

Future work could investigate further enhancements to the GA applied to the 2TP problem. One potential direction is the exploration of adaptive GAs, where parameters such as population size, mutation rate, and crossover probabilities adjust dynamically throughout the optimization process. Additionally, alternative selection mechanisms, such as rank-based or tournament selection with elitism, could be explored to improve convergence and prevent premature stagnation. Implementing an early stop criterion could also be beneficial, halting the algorithm when no significant improvement in fitness is observed over a number of generations, thus saving computational resources. Further, combining GAs with local search methods, such as hill-climbing or simulated annealing, could refine the solutions by intensifying the search around promising individuals. Another avenue for future research is to assess the performance of the GA on larger graphs, investigating its scalability and efficiency in solving 2TP problems in more complex scenarios. Finally, exploring multi-objective GAs to optimize additional criteria, such as minimizing travel time or balancing load between users, could provide more nuanced solutions for real-world applications.

Note that, in addition to the exact solution and GAs, other methods can be employed to solve the 2TP problem. Heuristic approaches, such as greedy algorithms, can be used to iteratively select candidate meeting and dropping points based on local optimizations, though they may not guarantee global optimality. In earlier research, a heuristic approach was proposed to optimize meeting points in ride-sharing scenarios within extensive multi-modal road networks [18], demonstrating its ability to find near-optimal solutions efficiently. Meta-heuristic methods, like simulated annealing or

particle swarm optimization, offer alternative search strategies that balance exploration and exploitation, potentially providing near-optimal solutions with reduced computational complexity. Furthermore, approximation algorithms could be applied to provide efficient solutions with bounded error in cases where exact solutions are too costly to compute. Further research could focus on extending the 2TP problem to include more than two users [19], as well as developing an exact algorithm that can compute solutions in subquadratic time.

VI. CONCLUSION

This paper introduced a Fitness Caching Genetic Algorithm (GA) to address the Two Transfer Point (2TP) problem, a critical optimization challenge in vehicle routing and ride-sharing. The GA demonstrated its ability to find near-optimal solutions for smaller problem instances and acceptable solutions for larger instances, efficiently handling scenarios where the exact algorithm could become computationally intractable. The GA's iterative population-based approach balances exploration and exploitation, effectively navigating the solution space while ensuring scalability. These results highlight the GA's potential as a practical heuristic for solving the 2TP problem. However, a trade-off remains between runtime and solution quality, particularly when the GA operates under the same execution time as the exact algorithm. Further research is necessary to fully evaluate its applicability and effectiveness in other real-world scenarios.

REFERENCES

- [1] K. H. Ting, L. S. Lee, S. Pickl, and H.-V. Seow, "Shared mobility problems: A systematic review on types, variants, characteristics, and solution approaches," *Applied Sciences*, vol. 11, no. 17, p. 7996, 2021.
- [2] F. Golpayegani, M. Gueriau, P.-A. Laharotte, S. Ghanadbashi, J. Guo, J. Geraghty, and S. Wang, "Intelligent shared mobility systems: A survey on whole system design requirements, challenges and future direction," *IEEE Access*, vol. 10, pp. 35 302–35 320, 2022, submitted 10 Oct 2022, Published 28 March 2022.
- [3] M. Zhu, X.-Y. Liu, and X. Wang, "An online ride-sharing path-planning strategy for public vehicle systems," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 2, pp. 616–627, 2018.
- [4] Y. Cao, S. Wang, and J. Li, "The optimization model of ride-sharing route for ride hailing considering both system optimization and user fairness," *Sustainability*, vol. 13, no. 902, p. 902, 2021.
- [5] H. Zhang, H. Ge, J. Yang, and et al., "Review of vehicle routing problems: Models, classification and solving algorithms," *Archives of Computational Methods in Engineering*, vol. 29, pp. 195–221, 2022. [Online]. Available: <https://doi.org/10.1007/s11831-021-09574-x>
- [6] N. Mardešić, T. Erdelić, T. Carić, and M. urasević, "Review of stochastic dynamic vehicle routing in the evolving urban logistics environment," *Mathematics*, vol. 12, no. 1, p. 28, 2024.
- [7] N. Alisoltani, M. Zargayouna, L. Leclercq, and M. Ameli, "Real-time autonomous taxi service: An agent-based simulation," *Agents and Multi-Agent Systems: Technologies and Applications*, pp. 199–207, 2020.
- [8] A. Manjunath, V. Raychoudhury, S. Saha, S. Kar, and A. Kamath, "Care-share: A cooperative and adaptive strategy for distributed taxi ride sharing," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 7, pp. 7028–7044, 2022.
- [9] E. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959. [Online]. Available: <http://eudml.org/doc/131436>
- [10] A. Gajentaan and M. H. Overmars, "On a class of $O(n^2)$ problems in computational geometry," *Computational Geometry*, vol. 45, no. 4, pp. 140–152, 2012. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925772111000927>
- [11] A. G. Jørgensen and S. Pettie, "Threesomes, degenerates, and love triangles," *CoRR*, vol. abs/1404.0799, 2014. [Online]. Available: <http://arxiv.org/abs/1404.0799>
- [12] A. Freund, "Improved subquadratic 3sum," *Algorithmica*, vol. 77, pp. 440–458, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:29179410>
- [13] D. M. Kane, S. Lovett, and S. Moran, "Near-optimal linear decision trees for k-sum and related problems," 2017. [Online]. Available: <https://arxiv.org/abs/1705.01720>
- [14] I. Abraham, D. Delling, A. Fiat, A. V. Goldberg, and R. F. Werneck, "Highway dimension and provably efficient shortest path algorithms," *J. ACM*, vol. 63, no. 5, Dec. 2016. [Online]. Available: <https://doi.org/10.1145/2985473>
- [15] R. Geisberger, P. Sanders, D. Schultes, and D. Delling, "Contraction hierarchies: Faster and simpler hierarchical routing in road networks," in *Experimental Algorithms*, ser. Lecture Notes in Computer Science, C. C. McGeoch, Ed., vol. 5038. Springer, Berlin, Heidelberg, 2008, pp. 319–333. [Online]. Available: https://doi.org/10.1007/978-3-540-68552-4_24
- [16] J. H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control and Artificial Intelligence*. Cambridge, MA, USA: MIT Press, 1992.
- [17] S. Patni and B. Sharma, "Genetic algorithms for decision optimization," in *Intelligent Decision Making Through Bio-Inspired Optimization*. IGI Global, 2024, p. 11.
- [18] J. Baudru and H. Bersini, "Heuristic optimal meeting point algorithm for car-sharing in large multimodal road networks," in *Proceedings of the 10th International Conference on Vehicle Technology and Intelligent Transport Systems. VEHITS*, 2024. [Online]. Available: <https://doi.org/10.5220/0000186800003702>
- [19] K. Liu, J. Liu, and J. Zhang, "Advanced algorithms for optimal meeting points in road networks," *IET Intelligent Transport Systems*, vol. 17, no. 2, pp. 221–231, 2022. [Online]. Available: <https://doi.org/10.1049/itr2.12323>